D4-V4 — Search & Rescue: Heterogeneous Sensing

Detailed project report — PDDL & PDDL+ — Artificial Intelligence for Robotics 2 (AI4RO2), 2026
Author: **Amr Magdy Mohamed Elsayed Abdalla** • ID: **S8082888**
Repository: https://github.com/amrmagdy16/AI2_Assignment

Abstract. This project models a multi-robot urban search-and-rescue (SAR) team in which sensing is *heterogeneous*: thermal, visual and gas sensors are distributed across different robots and no single robot owns all of them. A classical PDDL model (Q1) captures sensing as knowledge-producing actions and forces the robots to cooperate, while a PDDL+ model (Q2) adds a continuously evolving gas hazard through processes and events, coupling the sensing decisions to the dynamics. The report documents every modelling choice, walks through the generated plans (validated with BFWS for Q1 and ENHSP for Q2), and discusses the abstraction of perception and its interaction with continuous dynamics.

1. Introduction and problem statement

Robotic autonomy is commonly framed as a **Sense–Plan–Act** loop. Automated planning addresses the *Plan* stage: given a symbolic description of the world, an initial state and a goal, a domain-independent planner searches for a sequence of actions that achieves the goal. The recurring theme of the course is that planning is only possible because we deliberately **abstract** reality into something a search algorithm can manipulate — and that the power of any model comes precisely from what it chooses to hide.

The chosen assignment, **D4-V4: Search and Rescue – Heterogeneous Sensing**, makes that theme concrete. A team of mobile robots searches a partially damaged building, modelled as a graph of rooms connected by doorways. The robots carry different sensors:

- **thermal sensors** detect heat signatures (a possible victim);
- **visual sensors** identify victims (and, in principle, obstacles);
- **gas sensors** detect hazardous gas concentrations.

Crucially, **no single robot has all three sensors**. The team must perform two tasks — *detection* and *rescue* — under the constraint of incomplete individual sensing. The objective of this report is to show how that constraint is represented in PDDL, how it shapes the resulting plans, and how the model extends to a world in which the hazard itself changes over time.

The remainder of the report is organised as follows. Section 2 states the modelling approach and maps it to the assignment requirements. Sections 3 and 4 present the classical model and its two validated instances. Sections 5 and 6 present the PDDL+ model, its processes and events, and the ENHSP output. Section 7 covers planner selection, design alternatives and verification, and Sections 8 and 9 discuss the abstraction of perception, its interaction with the dynamics, the project's limitations, and directions for future work.

2. Modelling approach and requirement mapping

The model rests on three design principles. First, **sensing capabilities are predicates** attached to each robot (`has-thermal`, `has-visual`, `has-gas`), so a robot can only perform a sensing action it is physically equipped for. Second, **perception is modelled as knowledge production**: a sensing action does not change the physical world, it adds a belief predicate (`detected`, `identified`, `gas-checked`). Third, **task order is never written explicitly**; it emerges from the precondition structure, which is the declarative strength of PDDL. The table below maps each assignment requirement to the construct that satisfies it.

D4-V4 requirement	How it is satisfied in this project
Sensing capabilities explicit	Per-robot has-thermal / has-visual / has-gas predicates
Sensing not universally available	Specialised robots, one sensor each (Q1-B, all Q2)
Sensing affects planning decisions	rescue requires the full detected+identified+gas-checked chain
Q1: sensing-dependent actions	thermal-scan, visual-identify, gas-sweep gated by capability
Q1: trivial instance	One robot carrying all three sensors (Problem A)
Q1: sensor-specific reasoning	Three single-sensor robots that must converge (Problem B)
Q2: process for hazard evolution	gas-buildup (and gas-spreads event for propagation)
Q2: event for detection/failure	victim-lost failure event at a lethal threshold
Q2: sensing vs dynamic hazard	rescue tests the live gas-level; ventilation forces waiting
Discussion	Abstraction of sensing; perception-dynamics coupling (Sec. 8)

Table 1. Assignment requirements and the corresponding modelling decisions.

3. Classical PDDL model (Q1)

3.1 Types and predicates

Three object types are declared — `robot`, `location` and `victim`. Typing keeps the action schemas compact and prevents nonsensical groundings (for example, scanning a location as if it were a victim). The predicate vocabulary separates three concerns: the physical world (position and topology), the robots' fixed capabilities, and the knowledge accumulated by sensing.

```
(:predicates
  (at ?r - robot ?l - location)            ; robot position
  (adjacent ?l1 - location ?l2 - location) ; map edge (declared both ways)
  (victim-in ?v - victim ?l - location)    ; ground-truth victim position
  (has-thermal ?r - robot) (has-visual ?r - robot) (has-gas ?r - robot)
  (detected ?v - victim)      ; heat signature picked up
  (identified ?v - victim)    ; visually confirmed victim
  (gas-checked ?l - location) ; room swept for gas
  (rescued ?v - victim))
```

Note that `adjacent` is not symmetric by default: both directions must be asserted in the problem file, otherwise a robot could move only one way through a doorway — a classic PDDL navigation pitfall.

3.2 Actions

Five action schemas are defined. `move` is ordinary graph navigation. The three sensing actions are each guarded by the matching capability predicate, which is what makes the team heterogeneous. `thermal-scan` turns ground truth into a *detection*; `visual-identify` upgrades a detection into an *identification* (note it requires `detected` as a precondition, so the two sensors are causally chained); `gas-sweep` clears a room. Only when a victim is identified **and** its room is gas-checked can `rescue` fire.

```
(:action move
  :parameters (?r - robot ?from - location ?to - location)
  :precondition (and (at ?r ?from) (adjacent ?from ?to))
  :effect (and (not (at ?r ?from)) (at ?r ?to)))

(:action thermal-scan
  :parameters (?r - robot ?v - victim ?l - location)
  :precondition (and (at ?r ?l) (has-thermal ?r) (victim-in ?v ?l))
  :effect (and (detected ?v)))

(:action gas-sweep
  :parameters (?r - robot ?l - location)
  :precondition (and (at ?r ?l) (has-gas ?r))
```

```
  :effect (and (gas-checked ?l)))
```

The chaining is worth emphasising. `visual-identify` cannot run until a `thermal-scan` has produced `detected`, so the two perception steps are forced into a fixed causal order even though that order is never stated procedurally. The remaining two schemas, `visual-identify` and the goal-achieving `rescue`, complete the chain:

```
(:action visual-identify
  :parameters (?r - robot ?v - victim ?l - location)
  :precondition (and (at ?r ?l) (has-visual ?r) (victim-in ?v ?l) (detected ?v))
  :effect (and (identified ?v)))

(:action rescue
  :parameters (?r - robot ?v - victim ?l - location)
  :precondition (and (at ?r ?l) (victim-in ?v ?l) (identified ?v) (gas-checked ?l))
  :effect (and (rescued ?v)))
```

Because the sensing capabilities are split across robots and the rescue precondition references the products of all three sensors, the planner is **obliged** to route the right robot to the right room. The cooperation is not scripted; it is a logical consequence of the preconditions. `rescue` itself needs no sensor, modelling the idea that any robot can perform the extraction once the room has been understood.

4. Q1 problem instances and generated plans

4.1 Problem A — sensing trivial

One robot `r1` carries all three sensors; one victim lies in `roomA`. Sensing still imposes an *order*, but no coordination is needed because a single agent can do everything. BFWS returns the minimal five-action plan, which is exactly the sensing chain followed by the rescue:

```
(:objects r1 - robot   entrance roomA - location   v1 - victim)
(:init (at r1 entrance) (adjacent entrance roomA) (adjacent roomA entrance)
       (victim-in v1 roomA) (has-thermal r1) (has-visual r1) (has-gas r1))
(:goal (and (rescued v1)))
```

```
(move r1 entrance rooma)
(thermal-scan r1 v1 rooma)
(visual-identify r1 v1 rooma)
(gas-sweep r1 rooma)
(rescue r1 v1 rooma)
```

Read step by step: r1 walks into the room, the thermal sensor registers a heat signature, the visual sensor confirms it is a victim, the gas sensor declares the room safe, and only then is the victim extracted. This instance is the control case — it isolates the sensing *order* from any question of which robot owns which sensor.

4.2 Problem B — sensor-specific reasoning

Here three specialised robots carry one sensor each (`thermalbot`, `visualbot`, `gasbot`) and two victims sit in `roomA` and `roomB`, reached through a central `hallway`. No robot can complete the chain alone, so all three must travel to each room. A valid plan (online BFWS) is:

```
(:objects thermalbot visualbot gasbot - robot
          entrance hallway roomA roomB - location   v1 v2 - victim)
(:init (at thermalbot entrance) (at visualbot entrance) (at gasbot entrance)
       (adjacent entrance hallway) (adjacent hallway entrance)
       (adjacent hallway roomA) (adjacent roomA hallway)
       (adjacent hallway roomB) (adjacent roomB hallway)
       (victim-in v1 roomA) (victim-in v2 roomB)
       (has-thermal thermalbot) (has-visual visualbot) (has-gas gasbot))
(:goal (and (rescued v1) (rescued v2)))
```

```
(move thermalbot entrance hallway)
```

```
(move thermalbot hallway roomb)
(thermal-scan thermalbot v2 roomb)
(move visualbot entrance hallway)
(move gasbot entrance hallway)
(move visualbot hallway roomb)
(visual-identify visualbot v2 roomb)
(move gasbot hallway roomb)
(gas-sweep gasbot roomb)
(rescue gasbot v2 roomb)
(move thermalbot roomb hallway)
(move thermalbot hallway rooma)
(thermal-scan thermalbot v1 rooma)
(move visualbot roomb hallway)
(move gasbot roomb hallway)
(move gasbot hallway rooma)
(gas-sweep gasbot rooma)
(move visualbot hallway rooma)
(visual-identify visualbot v1 rooma)
(rescue gasbot v1 rooma)
```

The plan confirms the intended behaviour: the three robots independently converge on each victim's room, each contributing only the sensing action it is capable of, before a rescue is possible. The exact ordering may vary between planners (BFWS vs. the cloud solver), but the convergence pattern is invariant.

5. PDDL+ model (Q2): a dynamic gas hazard

Q1 assumes a frozen world. Q2 relaxes that assumption: toxic gas now **evolves continuously**, can spread between rooms, and can become lethal. This requires PDDL+, which introduces *processes* (continuous change driven by the world, not the planner) and *events* (discrete changes that fire automatically when a numeric condition becomes true). A single numeric fluent, `(gas-level ?l)`, tracks the gas concentration in each room.

Alongside the Q1 predicates, Q2 adds `alive` (a victim is still recoverable), `gas-source` / `no-gas-source` (whether a room is producing gas) and `venting` (an extractor is running), together with the numeric function and requirements:

```
(:functions (gas-level ?l - location))
(:requirements :strips :typing :numeric-fluents :time)
```

It is worth being precise about the three kinds of change PDDL+ distinguishes, because the report relies on the distinction. An **action** is chosen by the planner and applies an instantaneous, discrete effect. A **process** is not chosen by anyone: it runs automatically for as long as its precondition holds and applies continuous change proportional to elapsed time. An **event** is also world-driven, but it applies an instantaneous discrete effect the moment its numeric trigger becomes true. In this domain the planner controls the robots (actions), the gas obeys physics (processes), and thresholds such as spreading and lethality are crossed automatically (events). The planner's only levers over the continuous world are *when* to start ventilation and *how long* to wait.

5.1 Processes

Two processes model the competing dynamics. `gas-buildup` raises the level while a room is an active source; `gas-ventilation` lowers it while an extractor fan is running, stopping at zero. The term `(* #t k)` means the change is proportional to elapsed time at rate *k* — the closest standard PDDL gets to an ordinary differential equation.

```
(:process gas-buildup
  :parameters (?l - location)
  :precondition (and (gas-source ?l))
  :effect (and (increase (gas-level ?l) (* #t 1))))
```

```
(:process gas-ventilation
  :parameters (?l - location)
  :precondition (and (venting ?l) (&gt; (gas-level ?l) 0))
  :effect (and (decrease (gas-level ?l) (* #t 2))))
```

5.2 Events

`gas-spreads` models propagation: once a room reaches level 8, gas seeps into a clean neighbour. Setting `gas-source` and clearing `no-gas-source` on the neighbour deletes the event's own precondition, so it fires at most once per edge (the standard rule that an event must invalidate its trigger). `victim-lost` is the required **failure** event: a victim in a room at level 10 or above becomes unrecoverable.

```
(:event gas-spreads
  :parameters (?l1 ?l2 - location)
  :precondition (and (adjacent ?l1 ?l2) (&gt;= (gas-level ?l1) 8) (no-gas-source ?l2))
  :effect (and (gas-source ?l2) (not (no-gas-source ?l2))))

(:event victim-lost
  :parameters (?v - victim ?l - location)
  :precondition (and (victim-in ?v ?l) (alive ?v) (&gt;= (gas-level ?l) 10))
  :effect (and (not (alive ?v))))
```

5.3 Modified actions and the sensing–dynamics coupling

A new action `start-ventilation` lets a gas robot switch on the extractor (requiring it to have first swept the room). The decisive change is in `rescue`: besides the discrete `gas-checked` flag it now also tests the **live** numeric condition `(< (gas-level ?l) 5)` and that the victim is still `alive`.

```
(:action start-ventilation
  :parameters (?r - robot ?l - location)
  :precondition (and (at ?r ?l) (has-gas ?r) (gas-checked ?l))
  :effect (and (venting ?l)))

(:action rescue
  :parameters (?r - robot ?v - victim ?l - location)
  :precondition (and (at ?r ?l) (victim-in ?v ?l) (identified ?v)
                     (gas-checked ?l) (alive ?v) (&lt; (gas-level ?l) 5))
  :effect (and (rescued ?v) (not (victim-in ?v ?l))))
```

This single precondition is the heart of the assignment's Q2: a gas-sweep is only a *snapshot*, but the gas keeps changing, so a stale reading is insufficient. The planner must reason about **when** the room is safe, not merely whether it was once checked — perception and continuous dynamics are now coupled.

Quantity	Value	Meaning
buildup rate	+1 / time unit	gas-source rooms fill
ventilation rate	-2 / time unit	extractor empties a room
spread threshold	gas-level >= 8	gas seeps to a clean neighbour
safe-to-rescue	gas-level < 5	rescue precondition
lethal threshold	gas-level >= 10	victim-lost failure event

Table 2. Numeric thresholds governing the hazard dynamics.

6. Q2 problem instances and ENHSP output

6.1 Problem A — ventilation forces the planner to wait

`roomA` starts at gas level 6, above the safe-to-rescue limit of 5, with no active source. The robots complete all sensing at time 0, but rescue is illegal until the room is ventilated. ENHSP therefore turns on the fan and inserts an explicit waiting step; at rate 2 the level falls 6→4 in one time unit, after which

rescue becomes legal:

```
0: (move gasbot entrance hallway)
0: (move gasbot hallway roomA)
0: (gas-sweep gasbot roomA)
0: (move visualbot entrance hallway)
0: (move thermalbot entrance hallway)
0: (move thermalbot hallway roomA)
0: (move visualbot hallway roomA)
0: (thermal-scan thermalbot v1 roomA)
0: (start-ventilation gasbot roomA)
0: (visual-identify visualbot v1 roomA)
0: -----waiting---- [1.0]
1.0: (rescue thermalbot v1 roomA)
Plan-Length:12   Elapsed Time: 1.0   Problem Solved
```

The `-----waiting---- [1.0]` line is direct evidence that the continuous process drives the solution: the planner deliberately lets time pass so that the world (not an action) brings the gas into the safe range. This is the course principle that, under PDDL+, *doing nothing is itself a decision with consequences*.

6.2 Problem B — an active source during the mission

Two victims are present; `roomB` contains an active gas source that climbs from 0 while `roomA` again needs ventilation. The single gas robot must service both rooms. ENHSP solves it in 22 steps, again using a wait, and rescues both victims before any lethal threshold is reached:

```
0: (move gasbot entrance hallway) ... 0: (gas-sweep gasbot roomA)
0: (thermal-scan thermalbot v2 roomB)
0: (start-ventilation gasbot roomA)
0: -----waiting---- [1.0]
1.0: (gas-sweep gasbot roomB) ... 1.0: (visual-identify visualbot v2 roomB)
1.0: (rescue gasbot v2 roomB)
1.0: (rescue thermalbot v1 roomA)
Plan-Length:22   Problem Solved
```

The thresholds are deliberately survivable, so a valid plan exists and the failure/spread events do not fire. To demonstrate genuine infeasibility — in which the planner would have to prioritise one victim and lose the other — it is enough to lower the lethal threshold from 10 to about 6 or to raise the buildup rate, at which point the `victim-lost` event becomes unavoidable on the slower route. Both instances behave exactly as designed, confirming that the model is correct.

The timeline is instructive. At time 0 the gas robot sweeps and ventilates roomA while the thermal robot reaches roomB and detects v2; the planner then waits one unit, during which roomA's level falls into the safe range and roomB's source rises by only one unit (from 0 to 1, far below the spread threshold of 8 and the lethal threshold of 10). At time 1.0 the gas robot relocates to roomB, the remaining sensing is completed in both rooms, and both victims are rescued. Because the single gas robot is the shared bottleneck, the plan implicitly schedules its movements around the evolving hazard — a small but genuine instance of multi-robot coordination under time pressure.

7. Planner selection and reproduction

Q1 is plain classical/numeric planning and is solved with **BFWS** (online editor or VS Code). Q2 uses processes and events, which require **ENHSP**; OPTIC is unsuitable because it rejects conditional processes, and conversely ENHSP does not accept durative actions — which is why all agent actions here are instantaneous and time advances only through the processes. ENHSP is run as:

```
java -jar ~/enhsp/ENHSP-Public/enhsp-dist/enhsp.jar -o q2_domain.pddl -f q2_problem_ventilation.pddl
```

Symptom	Fix

:numeric-fluents parse error	replace with :fluents
class file version error	compile and run with the same Java (17+); here Java 21
OPTIC: process with no conditions	use ENHSP for Q2
online ENHSP crash	install ENHSP locally

Table 3. Common toolchain issues and their fixes.

7.1 Design alternatives considered

Several modelling choices were made deliberately over plausible alternatives. A **single combined scan** action (one operator that detects, identifies and clears at once) was rejected because it would collapse the three sensor types into one capability and destroy the heterogeneity the assignment is about. Modelling sensing as **conditional effects** (sensing only if a victim is present) was avoided to keep the domain within the feature set BFWS handles cleanly and to keep the causal chain explicit. Using **durative actions** for ventilation or motion was ruled out because ENHSP — mandatory for the processes and events — does not support them; continuous change is instead expressed through processes, which is the more faithful PDDL+ idiom anyway. Finally, **obstacles** (which the visual sensor could in principle detect) were abstracted away because the assignment's tasks are detection and rescue, and geometric obstacle reasoning belongs to motion planning rather than task planning — a boundary discussed in Section 8.

7.2 Verification and validation

Each instance was checked against its intended behaviour rather than merely run. For Q1 the trivial plan must be the five-step chain and the cooperative plan must show all three specialised robots entering every victim room; both hold. For Q2 the key acceptance test is the presence of the explicit waiting step: if rescue could be scheduled at time 0 the continuous model would be inert, so the `-----waiting---- [1.0]` line is the positive evidence that the ventilation process genuinely gates the rescue. ENHSP additionally reports grounding statistics (for the ventilation instance, 23 ground actions, one process and no active events; for the dynamic instance, more actions with several processes and events instantiated), confirming that the processes and events were parsed and grounded as intended rather than silently ignored. No dead-ends were reported, and both problems terminate with a 'Problem Solved' status.

8.1 Abstraction of sensing

Sensing is represented as a **deterministic, perfect and instantaneous** production of a knowledge predicate. A thermal scan always succeeds, never produces a false positive, and reveals ground truth that is in fact already present in the initial state. Real perception is none of these things: it is noisy, partial and probabilistic. Classical PDDL has no vocabulary for uncertainty, so the model silently equates 'the robot sensed X' with 'X is true'. A faithful treatment would move to a **partially observable** formulation (a POMDP / belief space), where actions update a probability distribution rather than a Boolean fact.

The same simplification appears in the goal. The team is asked to *rescue* victims whose true positions are encoded in `victim-in` from the outset; the sensing actions merely 'unlock' the right to act on information the model already contains. In a genuinely partially observable setting the robots would not know how many victims exist or where, and exploration itself would carry value — precisely the concern of the sibling assignment on unknown victim locations. Treating perception as the revelation of pre-encoded truth is the single largest abstraction in this project, and it is the price paid for staying inside tractable classical/PDDL+ planning.

8.2 Interaction between perception and dynamics

The PDDL+ model exposes a tension that the classical one cannot. Because the gas level changes continuously, a safety reading has a limited shelf life; the live numeric precondition on `rescue` ties the value of a perception to *when* it is used. The planner consequently reasons not only about which robot senses what, but about the timing of the whole operation — ventilating, waiting, and acting within a window. This is the practical meaning of perception interacting with dynamics: information decays, and acting on stale information can be unsafe.

This coupling also changes what 'optimal' means. In the classical model a shorter action sequence is strictly better; in the PDDL+ model the planner trades plan length against makespan, sometimes preferring to *wait* rather than act, because only the passage of time can satisfy a numeric precondition. The ventilation instance makes this visible: the twelve-step plan is not the shortest conceivable sequence of robot actions but the shortest *feasible* one given that the room cannot be entered until the gas has physically dispersed. Reasoning about feasibility under continuous change, rather than mere reachability, is exactly the capability that separates PDDL+ from classical planning.

8.3 Position within the course

The two models occupy adjacent rungs of the abstraction ladder developed throughout AI4RO2: a Boolean symbolic world (Q1), then numeric and continuous-time dynamics via processes and events (Q2). The natural next rungs — which this project deliberately does not climb — are belief-space planning under partial observability and, ultimately, learning a robust policy when the model itself is unavailable (reinforcement learning).

Seen this way, the gap between Q1 and Q2 is itself the lesson. Q1 answers the question 'is the goal reachable given who can sense what?'; Q2 answers the harder question 'is the goal still reachable once the world refuses to stand still?'. Each added layer — numbers, then time, then autonomous processes and events — buys descriptive power at the cost of a larger and harder search, which is the standing tension of automated planning: the closer the model moves to physics, the more faithfully it describes a rescue mission, and the more expensive it becomes to solve.

9. Limitations, future work and conclusion

- **Perfect perception.** No false positives/negatives, no observation uncertainty; a POMDP would be required.
- **Discrete spread.** Gas propagation is an edge event, not true spatial diffusion; a per-edge flux process would be closer to physics but far larger.
- **Costless motion.** Movement is instantaneous, so the only source of temporal pressure is the processes; modelling travel time (e.g. as a motion process) would make routing temporally meaningful.
- **Fire-and-forget ventilation.** Once started, the extractor runs without requiring the robot to remain, which is a convenient idealisation.
- **Sequential multi-robot.** Robots act in a single interleaved sequence rather than with true concurrency, which standard PDDL does not natively express.

Future work. Replacing knowledge predicates with belief variables and semantic attachments (binding external localisation/perception routines to fluents), introducing motion as a continuous process so that path length costs time, and scaling the map to require heuristic guidance would each move the model closer to a deployable SAR planner.

Conclusion. The project delivers a complete, validated answer to D4-V4. The classical model makes heterogeneous sensing the binding constraint and produces cooperative plans purely through preconditions; the PDDL+ model adds a continuously evolving gas hazard whose processes and

events couple sensing to timing, with the planner's explicit waiting step demonstrating the dynamics at work. Above all it illustrates the central lesson of the course: a planning model is a contract, and its power and its blind spots are two sides of the same abstraction.